

LABORATOIRE
BORDELAIS
DE RECHERCHE
EN INFORMATIQUE

LaBRI



Manuel Utilisateur

Version 1.0

FLORIAN GRENIER

MASTER INFORMATIQUE – INTELLIGENCE ARTIFICIELLE

Encadrants universitaires :

PASCAL DESBARATS
pascal.desbarats@u-bordeaux.fr

TIMEA CSENGERI
timea.csengeri@u-bordeaux.fr

1er avril 2025 – 31 août 2025

université
de BORDEAUX

Table des matières

1	Introduction	2
2	Architecture du projet	2
3	Guide d'installation	3
4	Génération du dataset	3
4.1	Principe	3
4.2	Utilisation	3
4.3	Résultats produits	3
4.4	Bonnes pratiques	4
5	Entraînement du modèle d'apprentissage automatique	4
5.1	Utilisation	4
5.2	Paramètres disponibles	4
5.3	Résultats produits	4
5.4	Métriques suivies	4

1 Introduction

Ce document constitue le manuel utilisateur du projet développé dans le cadre de mon stage de fin d'études. Il vise à guider pas à pas l'utilisateur dans l'exploitation des outils que j'ai conçus pour l'analyse de raies spectrales astronomiques par apprentissage automatique.

L'application se compose de deux modules principaux et complémentaires :

Module de génération du jeu de données

Ce premier module permet de créer et de préparer le jeu d'apprentissage. Il inclut les différentes étapes de génération de spectres synthétiques, de normalisation et d'annotation, afin de produire un ensemble de données cohérent et exploitable par les algorithmes d'apprentissage automatique.

Module d'apprentissage automatique

Le second module est dédié à l'entraînement, à l'évaluation et à l'utilisation de réseaux de neurones pour la reconnaissance automatique de signatures spectrales. L'utilisateur pourra y configurer les paramètres d'entraînement, charger les modèles sauvegardés, et analyser les résultats produits.

2 Architecture du projet

Voici le récapitulatif de l'architecture générale de l'application,

```
AstroIA
├── pretrained_models
│   ├── best_model.pth
│   └── results
├── requirements.txt
├── src
│   ├── main.py
│   ├── data
│   │   └── molecule
│   ├── model_Deep_Learning
│   │   ├── data_generate
│   │   ├── dataset.py
│   │   ├── generate_spectral.py
│   │   ├── main.py
│   │   ├── model.py
│   │   ├── pretraitement.py
│   │   ├── result.py
│   │   ├── train.py
│   └── spectral_spectral_analysis
│       ├── __init__.py
│       ├── analysis.py
│       ├── main.py
│       ├── traitement.py
│       └── utils.py
```

3 Guide d'installation

Cette application est développée en **Python 3.11.2**. Il est donc recommandé d'utiliser cette version afin de garantir son bon fonctionnement.

Nous allons créer un environnement virtuel Python afin d'éviter les conflits avec les bibliothèques que nous allons installer par la suite.

Pour créer un environnement, exécutez la commande suivante : `python -m venv <nom_environnement>`

Ensuite, pour activer l'environnement virtuel :

— Sous Linux/MacOS : `source <nom_environnement>/bin/activate`

— Sous Windows : `<nom_environnement>\Scripts\activate`

À ce stade, votre terminal (selon l'OS utilisé) affichera probablement le nom de votre environnement au début de chaque ligne (celui que vous avez choisi à la place de `<nom_environnement>`).

Nous allons maintenant installer les bibliothèques nécessaires. Pour cela, placez-vous dans le dossier `AstroIArem/src` et exécutez la commande : `pip install -r requirements.txt`

Cette commande installe l'ensemble des dépendances indispensables au fonctionnement de l'application.

À présent, nous sommes en capacité d'exécuter l'application.

4 Génération du dataset

L'une des premières étapes nécessaires avant l'entraînement du réseau de neurones est la génération du dataset. Ce module permet de produire des spectres synthétiques simulant des observations astronomiques, tout en leur associant des étiquettes indiquant la présence ou non des différentes molécules.

4.1 Principe

Le générateur de spectres reproduit le comportement d'un spectromètre en créant des signaux artificiels, auxquels peuvent être ajoutés différents types de bruit (bruit gaussien, pollution). Ces données sont ensuite normalisées et stockées dans des fichiers afin d'être directement utilisables pour l'apprentissage automatique.

4.2 Utilisation

Pour générer un dataset, il suffit de lancer la commande suivante dans le terminal dans le dossier `AstroIArem/src/model` :

```
python3 generate_spectral.py
```

Par défaut, les paramètres de génération (nombre de spectres, dossier de sortie, etc.) sont définis directement dans le fichier. L'utilisateur peut modifier ces valeurs en ouvrant le script et en ajustant les variables correspondantes.

4.3 Résultats produits

Après exécution, un dossier `data/` est créé et contient les fichiers suivants :

— `synthetic_spectra_batch_*.npy` : spectres générés au format NumPy (par défaut : sans bruit, avec bruit, avec bruit et pollution) ;

- `synthetic_labels_batch*.npy` : étiquettes associées aux spectres ;
- `frequencies.npy` : grille de fréquences utilisée pour la génération ;
- `molecule_names.txt` : liste des molécules prises en compte.

4.4 Bonnes pratiques

- Vérifiez que l'espace disque est suffisant (les fichiers peuvent être volumineux).
- Commencez par générer un petit dataset (par ex. quelques centaines de spectres) afin de valider la configuration et le pipeline de traitement.
- Conservez une copie des datasets générés pour pouvoir reproduire vos expériences et comparer les résultats entre différents modèles.

5 Entraînement du modèle d'apprentissage automatique

L'entraînement du modèle est la seconde étape de l'application. Il consiste à utiliser le dataset généré précédemment afin d'optimiser un réseau de neurones pour la détection automatique des molécules dans les spectres.

5.1 Utilisation

Pour lancer l'entraînement, placez-vous dans le dossier `AstroIArem/src/model` et exécutez :

```
python3 main.py
```

Par défaut, les principaux paramètres d'entraînement sont définis dans le fichier `main.py`. L'utilisateur peut les modifier directement dans le code pour adapter l'apprentissage à ses besoins.

5.2 Paramètres disponibles

- **epochs** : nombre de passages complets sur le dataset (par défaut : 50).
- **batch_size** : taille des lots de spectres traités simultanément (par défaut : 32).
- **dataset_path** : chemin du dataset à utiliser (par défaut : `./data/generate`).
- **output_path** : dossier où seront enregistrés le modèle entraîné et les résultats (par défaut : `./results/`).

5.3 Résultats produits

À l'issue de l'entraînement, plusieurs fichiers sont générés :

- **best_model.pth** : poids du modèle entraîné.
- **training_log.txt** : journal contenant la perte (loss) et les métriques par époque.
- Graphiques (courbes de loss et de métriques) enregistrés dans `./results/`.

5.4 Métriques suivies

Les métriques suivantes sont calculées automatiquement à chaque époque :

- **Loss** : erreur entre la prédiction et la vérité terrain.
- **F1-score** : mesure de la qualité globale de la classification multi-label.

- **Exact Match Accuracy (EMA)** : proportion de spectres correctement prédits sur l'ensemble des molécules.
- **Hamming Loss** : proportion moyenne d'étiquettes incorrectement prédites.
- **mAP (mean Average Precision)** : précision moyenne sur toutes les classes.

Ces métriques permettent d'évaluer à la fois la qualité générale du modèle et sa capacité à détecter les différentes molécules présentes dans les spectres.